

PROYECTO

STREET RACE X

MANUAL TÉCNICO

Arquitectura y
Backend

Actividad:

Actividad
Backend

Plataforma:

Competencia
Urbana

Fecha:

Mayo 2026

1. INTRODUCCIÓN

Este documento describe la arquitectura técnica y la implementación integral del backend de Street Race X. El sistema utiliza Clean Architecture para asegurar que la lógica de negocio permanezca independiente de las herramientas de persistencia y comunicación.

2. STACK TECNOLÓGICO

El núcleo del sistema utiliza tecnologías modernas para garantizar alto rendimiento:

- **Runtime:** Node.js con TypeScript.
- **Framework:** Express.js.
- **Base de Datos:** Firebase Firestore (NoSQL).
- **Comunicación:** Socket.io para procesos en tiempo real.
- **Documentación:** Swagger OpenAPI 3.0.

3. ARQUITECTURA DEL SISTEMA

El proyecto se organiza en capas:

- **Domain:** Entidades centrales y reglas de negocio puras.
- **Interfaces:** Definición de contratos y abstracciones de repositorios.
- **Use Cases:** Orquestación de la lógica de aplicación.
- **Infrastructure:** Implementaciones técnicas y controladores.

4. LÓGICA DE NEGOCIO CRÍTICA

- **Vehículos:** Máximo de 3 unidades por usuario. Solo un vehículo activo a la vez.
- **Emparejamiento:** Retos permitidos únicamente entre pilotos del mismo rango.
- **Puntuación:** Incremento de +10 puntos por victoria y reducción de -5 por derrota.

5. COMUNICACIÓN REAL-TIME

Se utiliza Socket.io para notificaciones instantáneas de retos y resultados, garantizando una experiencia de usuario fluida y competitiva.

6. DOCUMENTACIÓN DE API (ENDPOINTS SWAGGER)

A continuación se detallan todos los endpoints disponibles en el servidor, tal como se visualizan en la documentación técnica de Swagger UI.

6.1. Autenticación (Auth)

MÉTODO	RUTA	DESCRIPCIÓN
POST	/api/auth/register	Registrar un nuevo piloto en el sistema.
POST	/api/auth/login	Iniciar sesión y obtener token de acceso.

6.2. Usuarios (Users)

MÉTODO	RUTA	DESCRIPCIÓN
GET	/api/users/me	Obtener la información del perfil del usuario autenticado.
GET	/api/users/discover	Buscar pilotos disponibles del mismo rango para retar.

6.3. Vehículos (Vehicles)

MÉTODO	RUTA	DESCRIPCIÓN
POST	/api/vehicles	Agregar un nuevo vehículo al garaje (máximo 3).
GET	/api/vehicles	Obtener la lista de vehículos del usuario.
PATCH	/api/vehicles/{id}/activate	Establecer un vehículo como activo para competir.

6.4. Retos (Challenges)

MÉTODO	RUTA	DESCRIPCIÓN
POST	/api/challenges	Enviar una solicitud de reto a otro piloto.
PATCH	/api/challenges/{id}/respond	Responder (Aceptar/Rechazar) a un reto recibido.
POST	/api/challenges/{id}/complete	Finalizar un reto y registrar al ganador.
GET	/api/challenges/history	Obtener el historial de retos del usuario.

6.5. Notificaciones (Notifications)

MÉTODO	RUTA	DESCRIPCIÓN
GET	/api/notifications	Obtener todas las notificaciones del usuario.
PATCH	/api/notifications/{id}/read	Marcar una notificación específica como leída.